

Vision-Guided Autonomous Mobile Manipulation for Assistive Robotics

Table of Contents

1. **Project Overview:** Mission & Objectives
2. **System Architecture:** Hardware Integration
3. **Workspace Setup:** ArUco Marker Mapping & Coordinate System
4. **Operational Workflow:** The Navigation & Decision Logic
5. **Perception Layer:** Overhead vs. Front-Facing Vision
6. **Distance & Sensor Logic:** Precision Positioning
7. **ARM Control and Grabbing**
8. **System Validation & Results**

Project Overview

An autonomous mobile platform powered by a **Raspberry Pi 4B (8GB)** integrating sonar, 1 overhead camera , one pie camera and Motor driver.

It utilizes a **6-DOF robotic arm** and **dual-motor** chassis for precise navigation and manipulation.

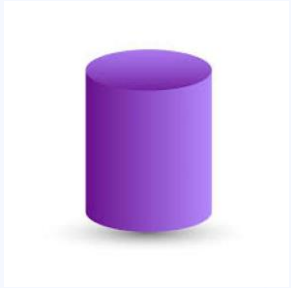
The system bridges global overhead vision with local sensor data for high-fidelity task execution.

System Architecture

Layer	Components	Function
Input	Overhead Cam, Pi Cam, Sonar(HC-SR04)	Data acquisition & Environment Mapping
Processing	Raspberry Pi 4B(8 Gb)	Object detection, Classification, & Path Logic, motor Command
Control	Motor Drivers Car motor:L298N Arm Motor:PCA9685	Translating logic into physical movement
Output	Car Motors & 6-DOF Arm	Navigation and Object Manipulation

Workspace Set Up:

A large **Rectangle**





- 2. The ArUco Network (Total: 6 Markers)



- **4 Corner Markers:** Define the global coordinate space and digital Boundary
- 1 Robo marker mounted on car to track real time's position
- 1 Destination marker: Placed at a fixed

3. Scatter Colored object (Triangles, Rectangles) around the table

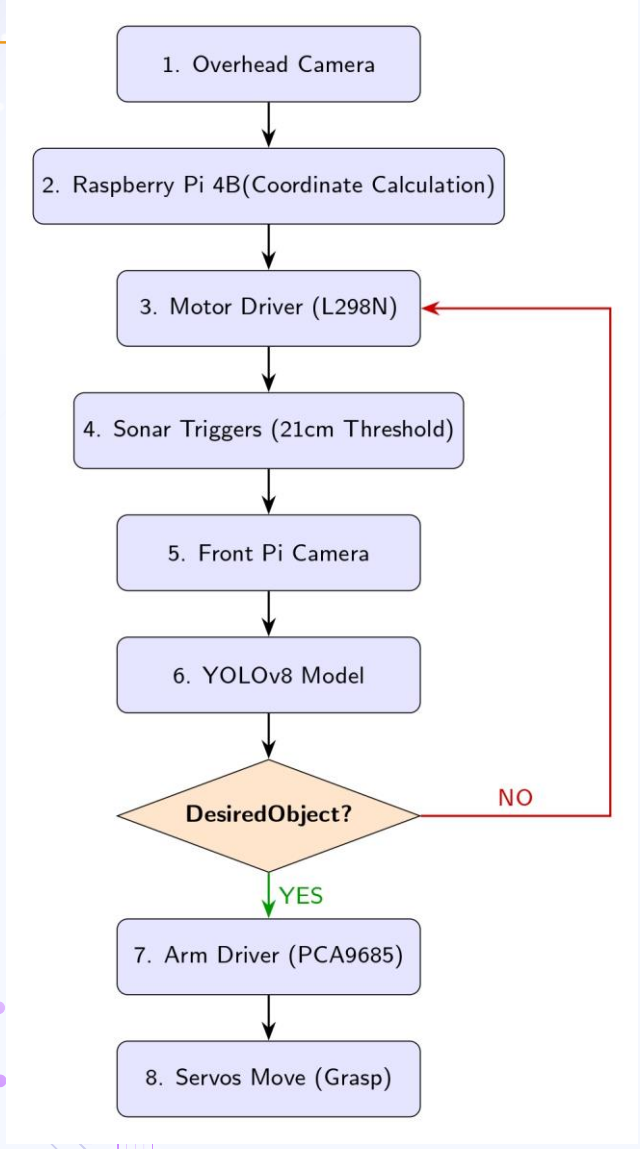


***ArUco markers are computationally lightweight for the Pi).**

Complete Workspace :



Overview of Operational workflow



- **Global-to-Local Transition**: Overhead vision guides the robot until the sonar confirms the 21cm hard stop, triggering the front camera.
- **Iterative Search (Fail-Safe):** The YOLOv8 decision node ensures the robot never wastes an arm movement on an undesired shape. It logs the invalid coordinate and immediately recalculates the path to the next nearest object (Step 3).
- **Closed-Loop Action**: Once validated, the Raspberry Pi executes the pre-calibrated 6-DOF sequence via the PCA9685 driver.

Operational workflow

Global Mapping and Initialization

- Overhead camera Capture Top Down View
- **Identifies Object** and **assign co-ordinate** to all object
- calculates the **Euclidean distance** to all detected objects and selects the closest coordinate as the primary target.

Targeted Approach & Precision Positioning

- **Mobile Navigation:** The car chassis moves toward the target coordinate to maintain its heading.
- **The 32 cm Threshold:** The robot monitors the distance using the front-mounted **Ultrasonic Sonar**.
- It executes a "Hard Stop" when the sonar detects the object at exactly **21 cm** (totaling 32 cm from the robot's center/marker).
- **Stability Lock:** The motors engage a braking state to ensure a steady frame for the next vision stage.

Operational workflow

Local Verification and (The decision Loop)

- ***Object Classification***: The front-facing Pi Camera activates AI Model 2.
- ***Validation Logic:***
- 1. IF "Desired": The robot triggers the 6-DOF arm pick-up sequence.
- 2. IF "Undesired": The robot logs this coordinate as "Completed/Invalid" and immediately pivots to the 2nd nearest coordinate identified in Phase 1.
- ***Iterative Search:*** This loop continues until a valid object is secured.

Manipulation and Delivery

- ***6-DOF Execution***: The robotic hand performs a pre-programmed trajectory to grasp the specific geometric shape (Prism, Pyramid, etc.).
- ***Homing Sequence***: The car identifies the Destination ArUco Marker.
- ***Deposit***: The robot navigates to the destination, the arm executes the "Release" command, and the system resets to find the next object.

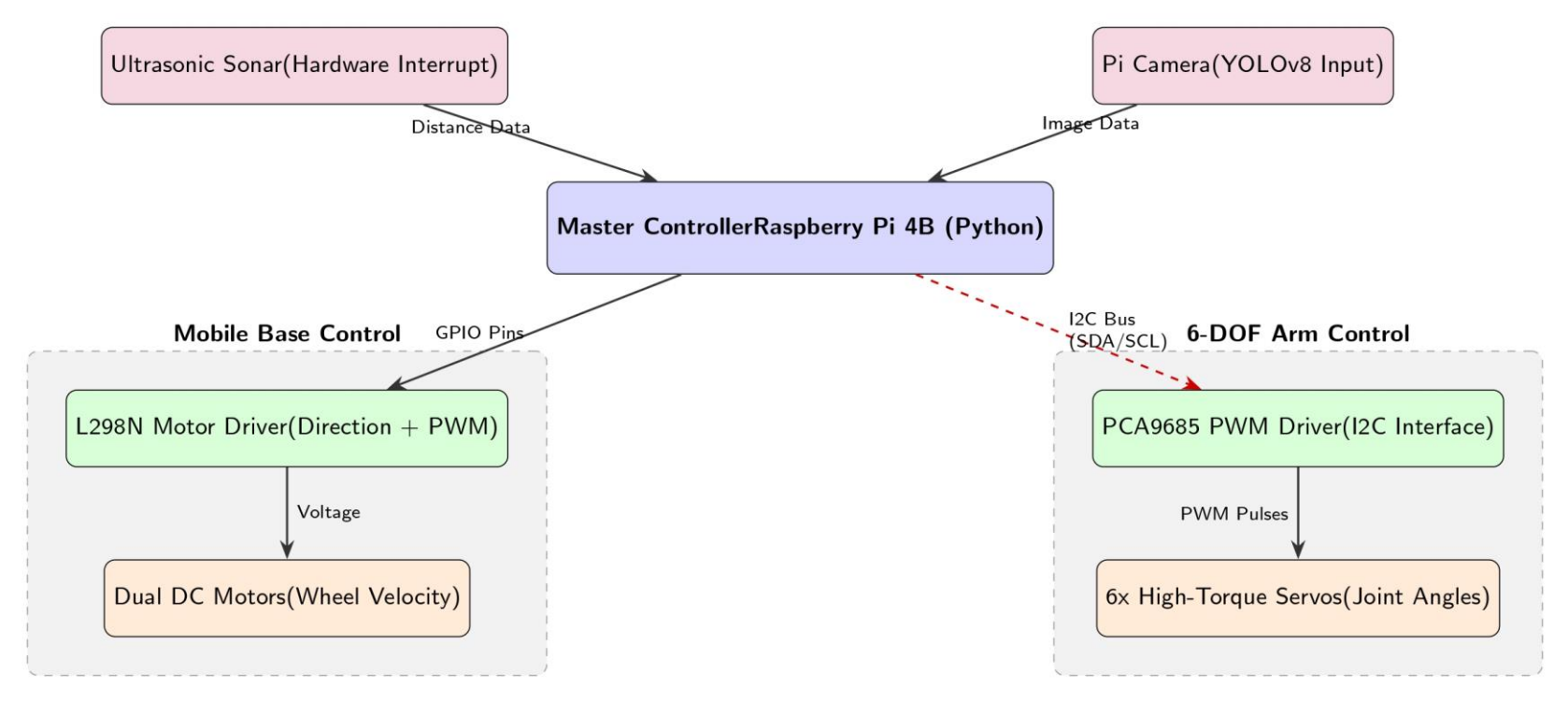
Operational workflow(The YOLO/Front Camera part)

Robot Exclusion Radius: The code calculates the Euclidean distance from the robot's ArUco center to the detected object. It ignores any YOLO detection within a 250 mm radius to prevent the robot from trying to pick up its own chassis.

Target Deduplication: If YOLO detects the same object twice (double bounding boxes), the code merges them if their centers are within 100 mm of each other.

Dynamic Sorting: Once validated, the target list is automatically sorted by distance, ensuring the robot always navigates to the closest object first to save time.

Hardware &System Control Architecture



I2C Fault Tolerance: The Raspberry Pi communicates with the PCA9685 arm driver over I2C. Because robotic arm movements draw high current, we implemented a 3-stage retry loop (try-except) in the Python code to catch I2C signal drops and prevent the arm from freezing mid-grasp.

Perception Layer

Overhead Vision (Global Perception):

The overhead camera acts as the **"GPS"** for your robot. It looks at the entire workspace from above to establish a common frame of reference.

Primary Task: Object Presence & Localization.

Model 1: Processes the wide-angle frame to identify regions where objects (prisms, cylinders, etc.) exist and assigns them (x, y) values.

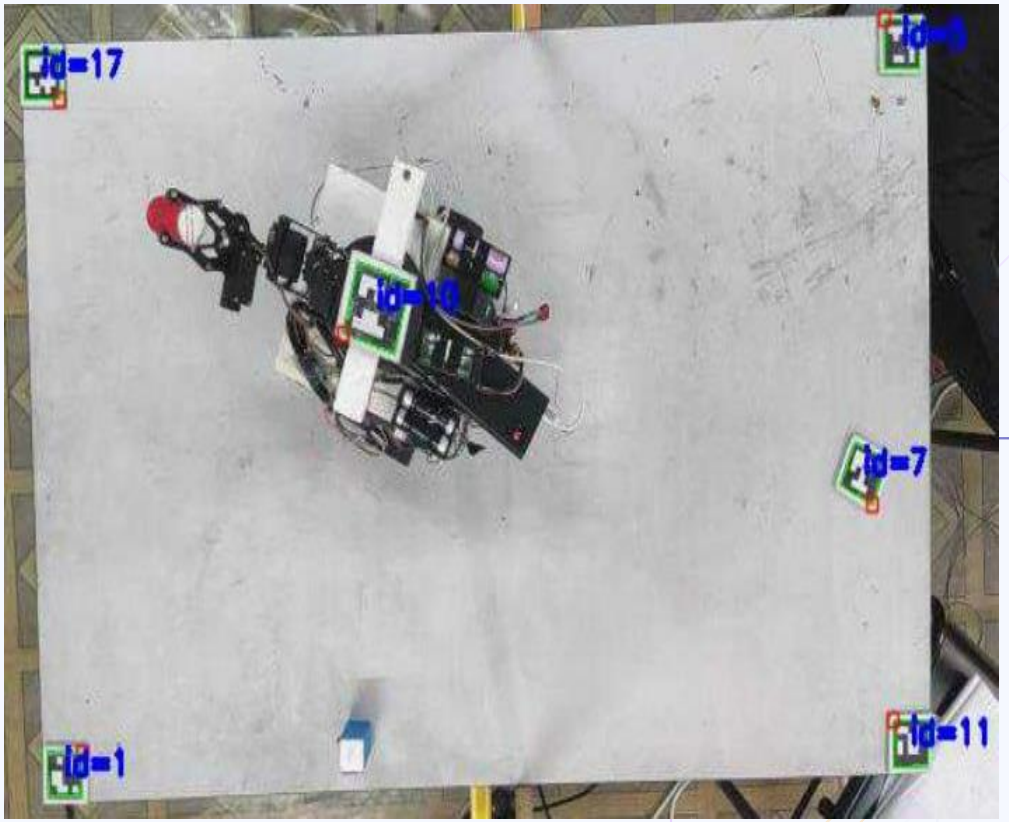
Mechanism: Detects ArUco markers to define the table boundaries and tracks the robot's coordinates relative to those corners.

Function: It tells the robot "Where things are" but not necessarily "What they are.

Model 1:Overhead detection







Perception Layer

Front-Facing Vision (Local Perception)

The Pi Camera mounted on the car acts as the "**Eyes**" of the robot. It only activates once the robot has reached its targeted coordinate.

Primary Task: Object Classification.

Mechanism: Captures a high-detail, close-up image of the object from a 32 cm distance.

Model 2: Analyzes the specific geometry and color of the object to distinguish between a desired shape (e.g., a Triangular Prism) and an undesired one (e.g., a Cylinder).

Function: It tells the robot "What the object is" so it can decide whether to use the 6-DOF arm or move to the next target.

YOLOv8 Training & Evaluation

Architecture: YOLOv8 (Nano/Small)

Epochs: **30** (Converged smoothly)

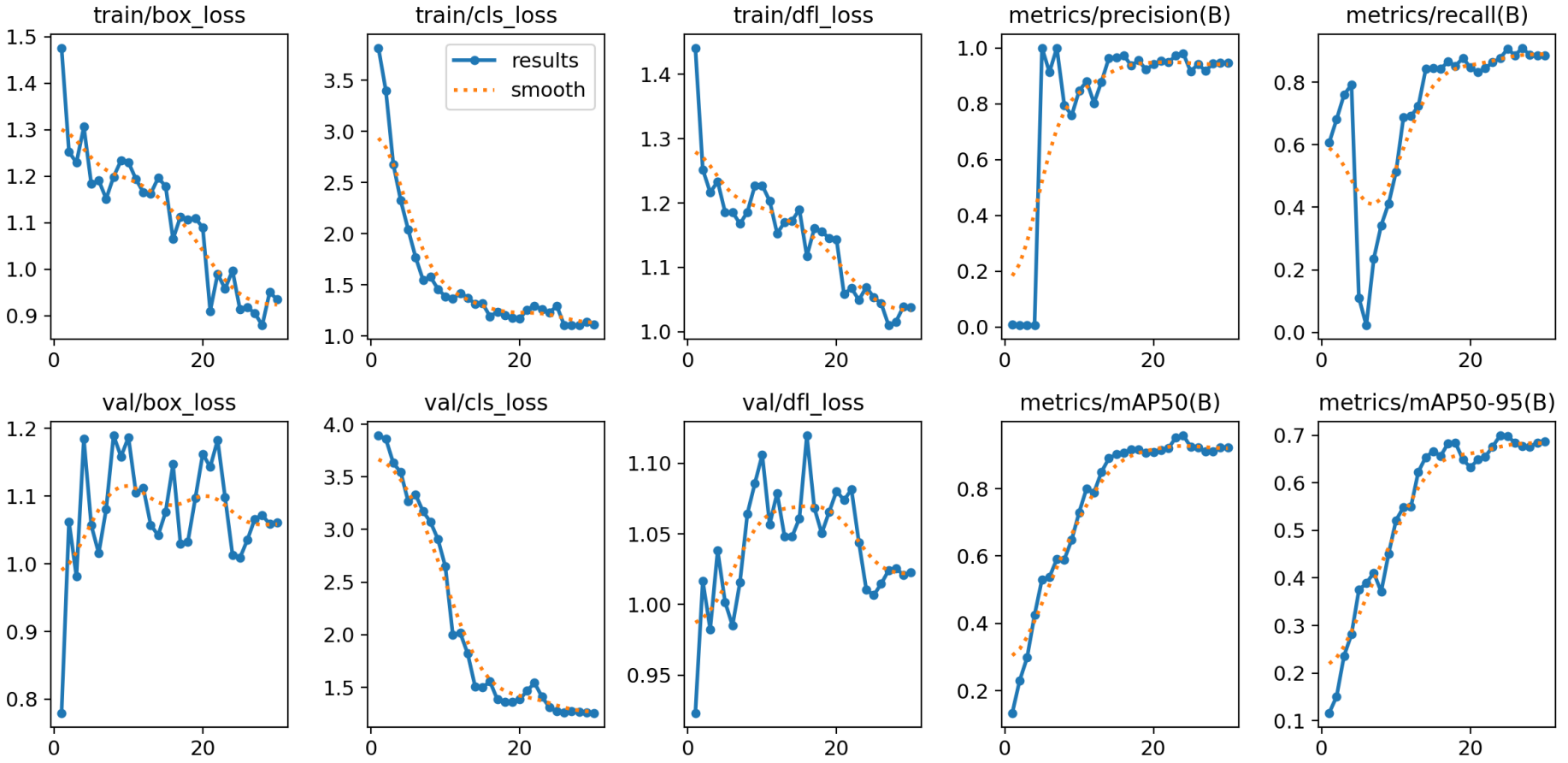
Best mAP@50: 95.8% (Achieved at Epoch 24)

Best mAP@50-95: 69.9% (Achieved at Epoch 24)

Final Precision: 94.6% (The robot rarely makes "False Positive" mistakes. It won't try to grab a cylinder if it thinks it's a desired prism)

Final Recall: 88.6% (The robot successfully finds almost all the desired objects in the frame.)

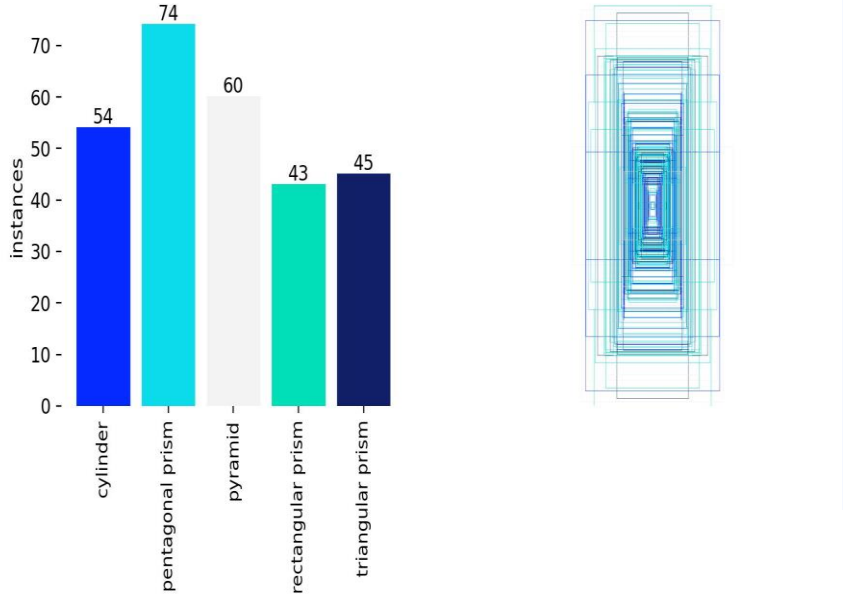
YOLOv8 Training & Evaluation

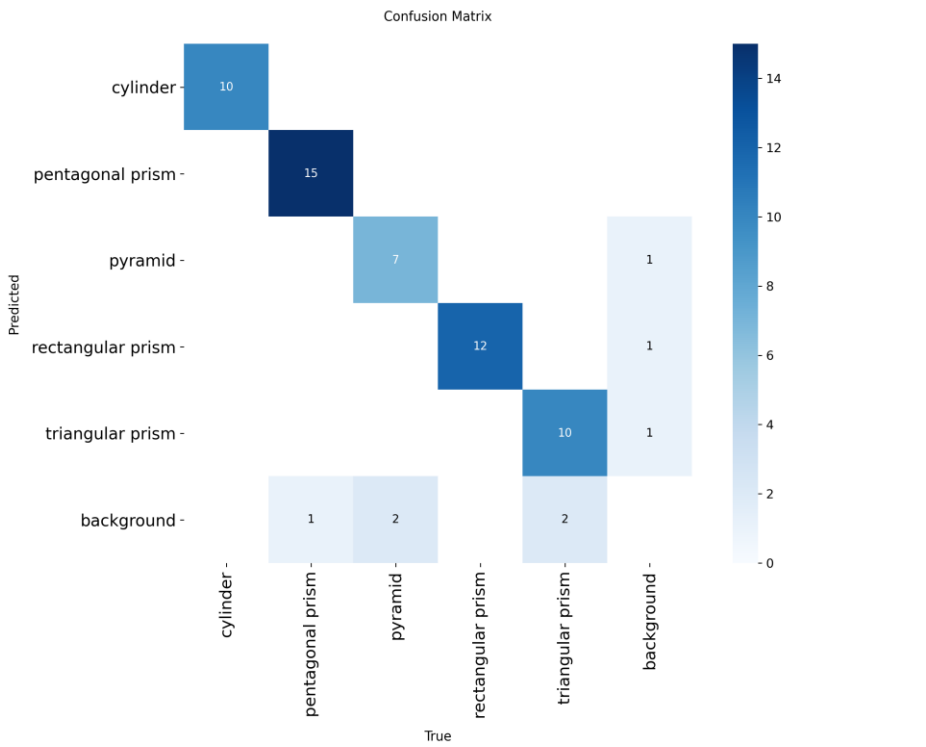


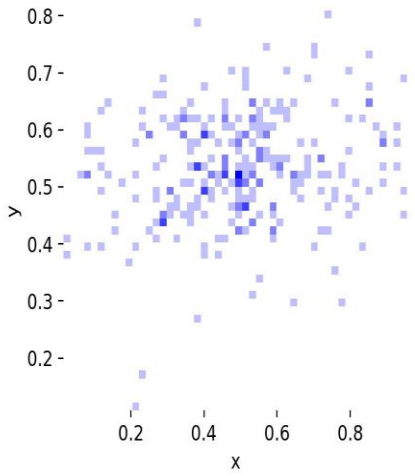
No Overfitting: Validation loss (val/cls_loss) closely tracks training loss, proving the model generalizes well to new objects.

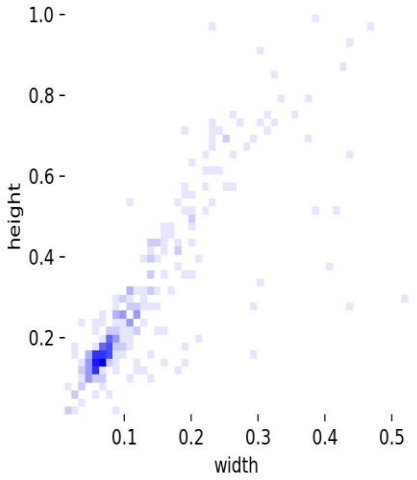
High Reliability: The steep climb in mAP50 and Precision after Epoch 10 means the robot reaches high confidence quickly, reducing false grabs.

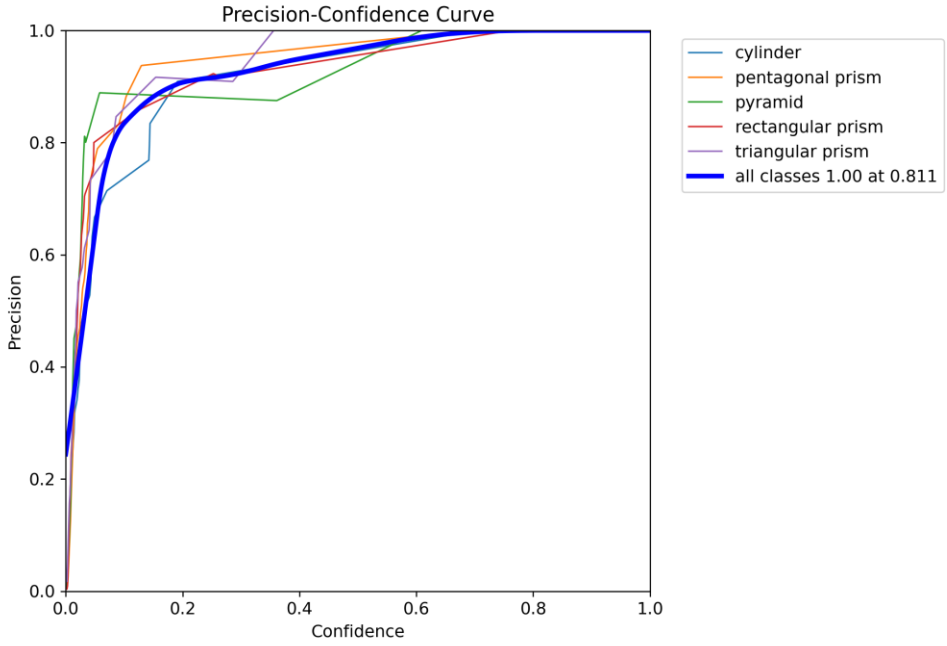
Performance of Model-2:



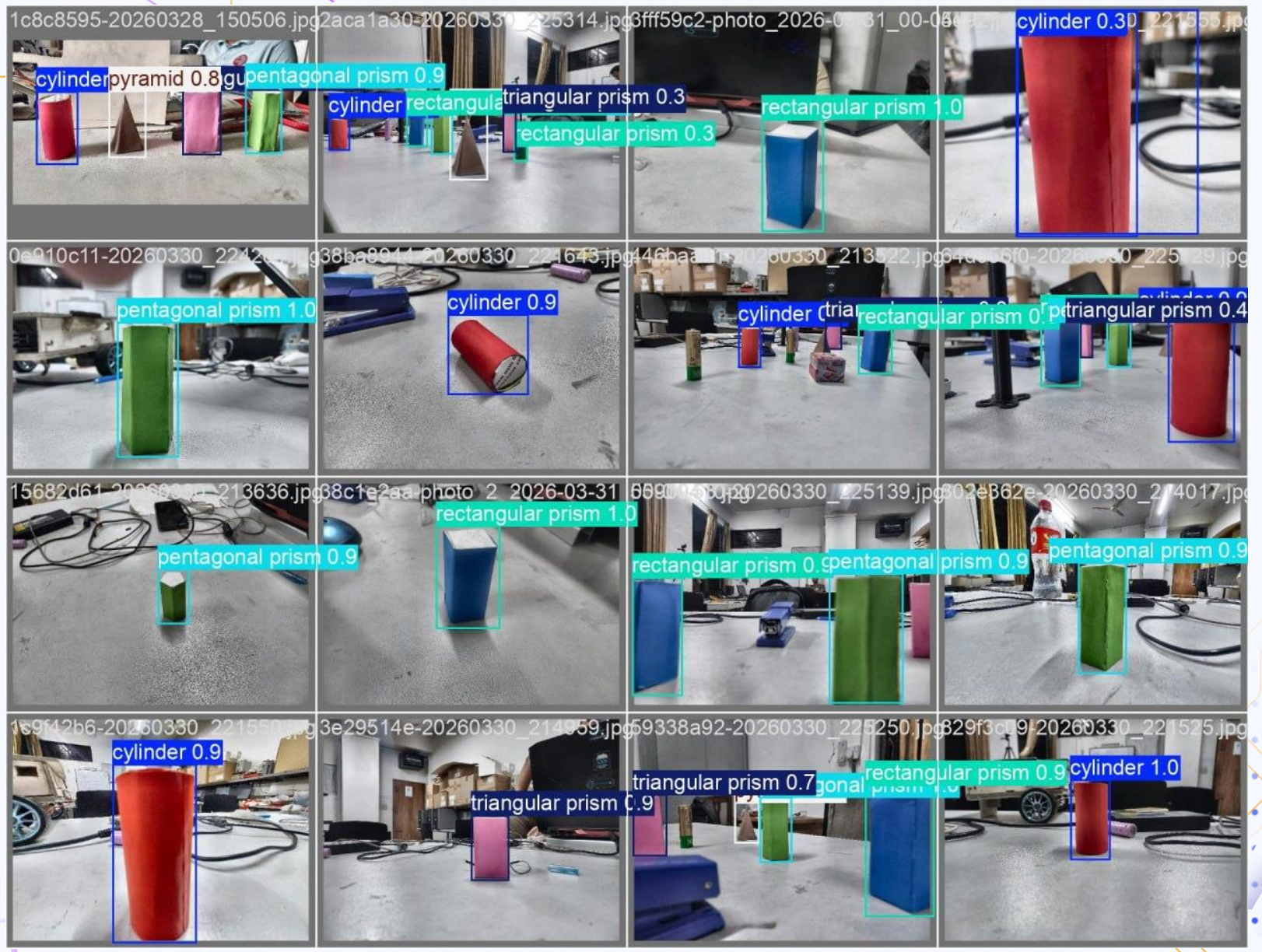








Model 2:Front face detection



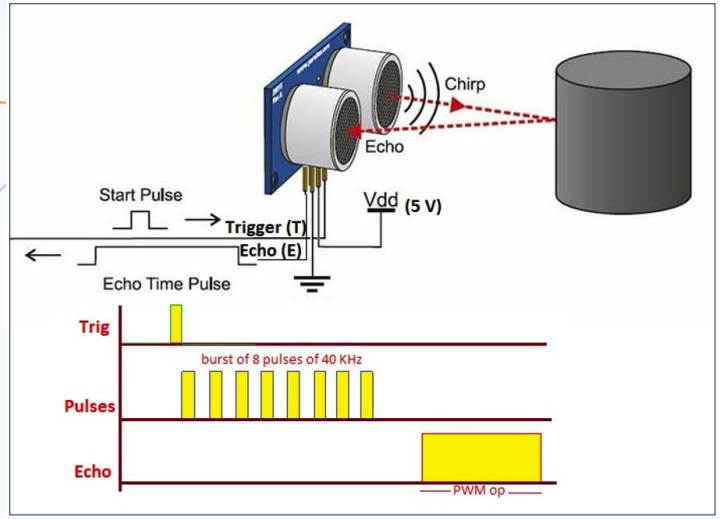
Distance & Sensor Logic: Precision Positioning

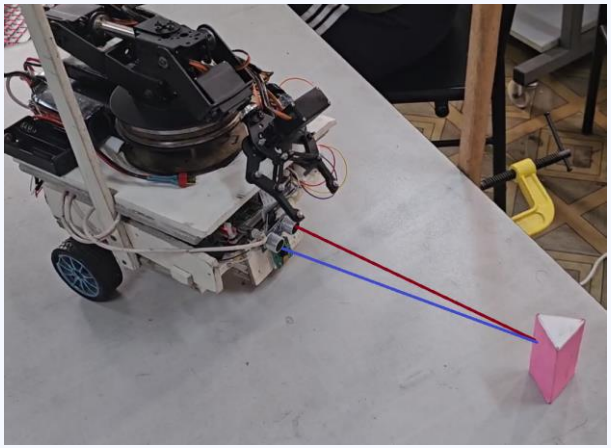
1. The Hardware Offset Configuration

To ensure the robotic arm is perfectly positioned for a "pick," the system calculates the distance from the robot's center .

Parameter	Value	Role
Arm Base to Sonar	11 cm	Internal Hardware Offset
Sonar to Object	21 cm	Target Operational Gap
Total System Gap	32 cm	Final Approach Threshold
Error Tolerance	±0.5 cm	Positioning Precision
Settle time	0.2 sec	System wait time after every movement

Distance & Sensor Logic: Precision Positioning





Sensor Noise Filtering (Deadband): The system ignores sonar readings outside the 20mm to 2000mm range to prevent false triggers from acoustic noise or table reflections (if 20 <= dist_mm <= 2000).

Cross-Verification (Camera + Sonar): The robot doesn't just blindly drive forward. It uses the Front Camera to find the bounding box center of the object, aligns left/right until the object is within a ±7 pixel tolerance, and only then uses the sonar to drive forward/backward until it hits the exact 220 mm target distance

DOF Robotic Arm Manipulation Sequence

The "Sense-to-Act" Transition:

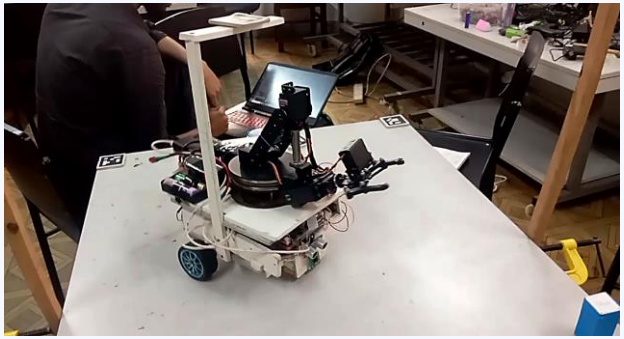
Model 2 Verifies Desired Object → PCA9685 PWM Driver manages 6 High torque Servos → Execute Pre-programmed multi step trajectory

Moto Name	Pulse to Angle Calibration
0.Base	(200, 300)
1. Shoulder	(275, 450)
2. Elbow	(300, 450)
3. Wrist Pitch	(100, 210)
4. Wrist Roll	(230, 260
5. Gripper	(100, 350)

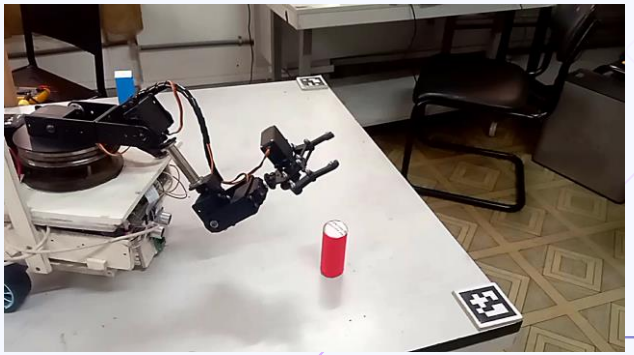
```
SEQUENCE = [
    # (name,                    [M0,  M1,  M2,  M3,  M4,  M5 ], delay_after)
    ("Base Position home",      [250, 330, 450, 100, 130, 150], 1.0),
    ("Base Position home1",     [250, 330, 300, 100, 130, 150], 3.0),
    ("Ground position",         [250, 450, 250, 100, 130, 150], 3.0),
    ("Before grab",             [250, 450, 250, 160, 130, 150], 3.0),
    ("After grab (grip close)", [250, 450, 250, 160, 130, 350], 3.0),
    ("After grab home1 (lift)", [250, 300, 250, 160, 130, 350], 3.0),
    ("After grab home2",        [250, 300, 250, 100, 130, 350], 3.0),
    ("After grab home3",        [250, 300, 400, 100, 130, 350], 3.0),
    ("Base Position (holding)", [250, 330, 450, 100, 130, 350], 3.0),
]
```

DOF Robotic Arm Manipulation Sequence

- *"Base Position home",*
- *"Base Position home1",*
- *"Ground position",*
- *"Before grab",*
- *"After grab (grip close)",*
- *"After grab home1 (lift)",*
- *"After grab home2",*
- *"After grab home3",*
- *"Base Position (holding)",*

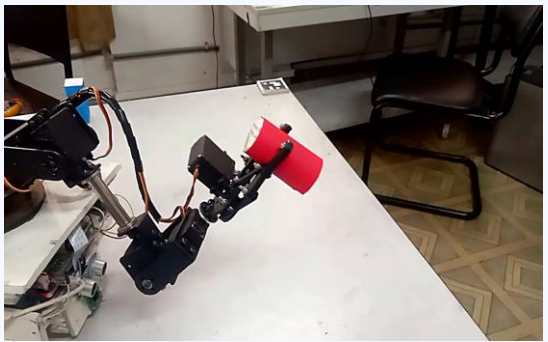


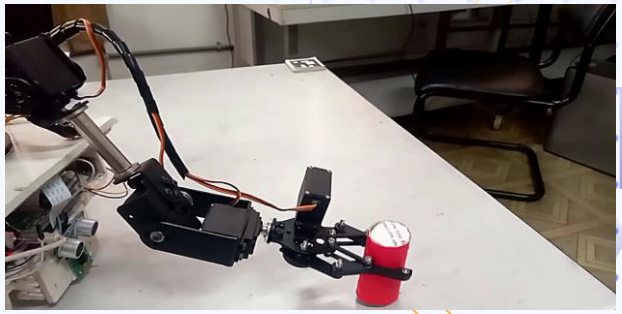






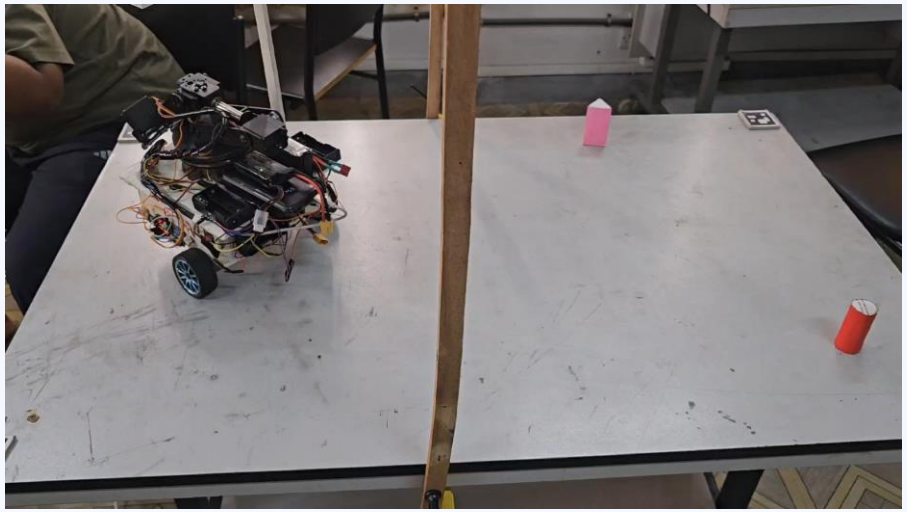


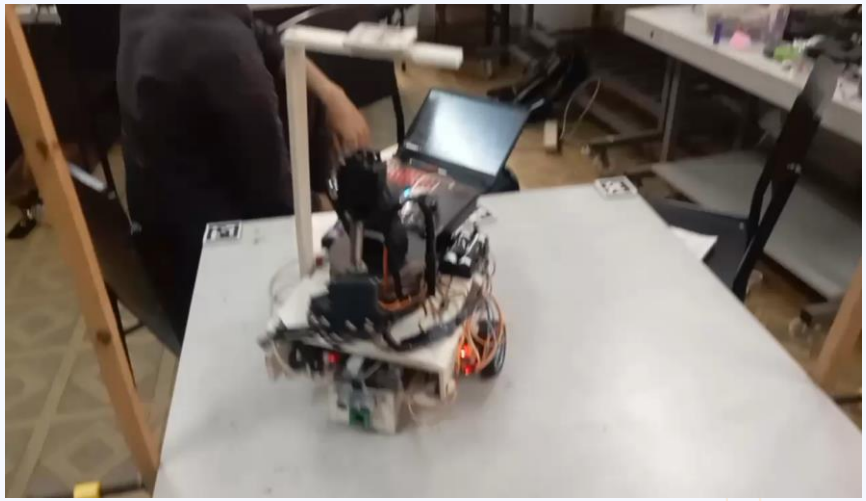




Final Submission

Youtube Link: https://youtu.be/w_YKosjn3Co





Thanks!

Do you have any questions?

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**